Class 13

May Alsoufi (PID: A17423250)

Table of contents

Background 1
Bioconductor setup 1
DESeq2 Required Inputs 4
Import countData and colData 4
Toy differential gene expression 6
Setting up for DESeq 11

Background

The data for this hands-on session comes from a published RNA-seq experiment where airway smooth muscle cells were treated with dexamethasone, a synthetic glucocorticoid steroid with anti-inflammatory effects

Bioconductor setup

As we already noted in a previous class Bioconductor is a large repository and resource for R packages that focus on analysis of high-throughput genomic data.

Recall that Bioconductor packages are installed differently than "regular" R packages from CRAN. To install the core Bioconductor packages, copy and paste the following two lines of code into your R console one at a time.

Check that you have installed everything correctly by entering the following two commands at the console window:

```
library(BiocManager)
library(DESeq2)
```

```
Loading required package: S4Vectors

Loading required package: stats4

Loading required package: BiocGenerics

Loading required package: generics


Attaching package: 'generics'

The following objects are masked from 'package:base':

    as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
    setequal, union


Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

    IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

    anyDuplicated, aperm, append, as.data.frame, basename, cbind,
    colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
    get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
    order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
    rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
    unsplit, which.max, which.min


Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

    findMatches
```

```
The following objects are masked from 'package:base':

    expand.grid, I, unname

Loading required package: IRanges

Loading required package: GenomicRanges

Loading required package: Seqinfo

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats


Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

    colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,
    colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
    colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
    colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
    colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
    colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
    colWeightedMeans, colWeightedMedians, colWeightedSds,
    colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
    rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
    rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
    rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
    rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
    rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
    rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
    rowWeightedSds, rowWeightedVars

Loading required package: Biobase
```

```
Welcome to Bioconductor

    Vignettes contain introductory material; view with
    'browseVignettes()'. To cite Bioconductor, see
    'citation("Biobase")', and for packages 'citation("pkgname")'.


Attaching package: 'Biobase'


The following object is masked from 'package:MatrixGenerics':

    rowMedians


The following objects are masked from 'package:matrixStats':

    anyMissing, rowMedians
```

DESeq2 Required Inputs

As input, the DESeq2 package expects
- a data.frame of count data (as obtained from RNA-seq or another high-throughput sequencing experiment) and - a second data.frame with information about the samples - often called sample metadata

The count matrix (called the `countData` in DESeq2-speak) the value in the i-th row and the j-th column of the data.frame tells us how many reads can be assigned to gene i in sample j.

For the sample metadata (i.e. `colData` in DESeq2-speak) samples are in rows and metadata about those samples are in columns.

Import countData and colData

```r
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
head(counts)
```

```
                SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
ENSG00000000003        723        486        904        445       1170
ENSG00000000005          0          0          0          0          0
ENSG00000000419        467        523        616        371        582
ENSG00000000457        347        258        364        237        318
```

```
ENSG00000000460         96         81         73         66        118
ENSG00000000938          0          0          1          0          2
                 SRR1039517 SRR1039520 SRR1039521
ENSG00000000003       1097        806        604
ENSG00000000005          0          0          0
ENSG00000000419        781        417        509
ENSG00000000457        447        330        324
ENSG00000000460         94        102         74
ENSG00000000938          0          0          0
```

Q1. How many genes are in this dataset?

```r
nrow(counts)
```

```
[1] 38694
```

There are 38694 genes in this dataset.

Q2. How many control cell lines do we have?

```r
metadata <- read.csv("airway_metadata.csv")
colnames(metadata)
```

```
[1] "id"       "dex"      "celltype" "geo_id"
```

```r
table(metadata$dex)
```

```
control treated
      4       4
```

There are 4 control samples.

Toy differential gene expression

```r
library(dplyr)
```

```
Attaching package: 'dplyr'

The following object is masked from 'package:Biobase':

    combine

The following object is masked from 'package:matrixStats':

    count

The following objects are masked from 'package:GenomicRanges':

    intersect, setdiff, union

The following object is masked from 'package:Seqinfo':

    intersect

The following objects are masked from 'package:IRanges':

    collapse, desc, intersect, setdiff, slice, union

The following objects are masked from 'package:S4Vectors':

    first, intersect, rename, setdiff, setequal, union

The following objects are masked from 'package:BiocGenerics':

    combine, intersect, setdiff, setequal, union

The following object is masked from 'package:generics':

    explain
```

```
The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
control <- metadata %>% filter(dex=="control")
control.counts <- counts %>% select(control$id)
control.mean <- rowSums(control.counts)/4
head(control.mean)
```

```
ENSG00000000003 ENSG00000000005 ENSG00000000419 ENSG00000000457 ENSG00000000460
         900.75            0.00          520.50          339.75           97.25
ENSG00000000938
           0.75
```

Instead of comparing 4 separate control columns, we summarize them into one number per gene. What this does: - Finds sample names that are control. - Extracts those columns from the counts table. - Calculates the average expression for each gene in control samples.

> Q3. How would you make the above code in either approach more robust? Is there a function that could help here?

Instead of using `control.mean <- rowSums(control.counts)/4`, use:

```r
control.mean <- rowMeans(control.counts)
```

It is better because it: - Automatically adjusts if number of control samples changes - Prevents hardcoding errors - Cleaner and safer

Now, we compare Treated vs Control… first we must calculate treated mean

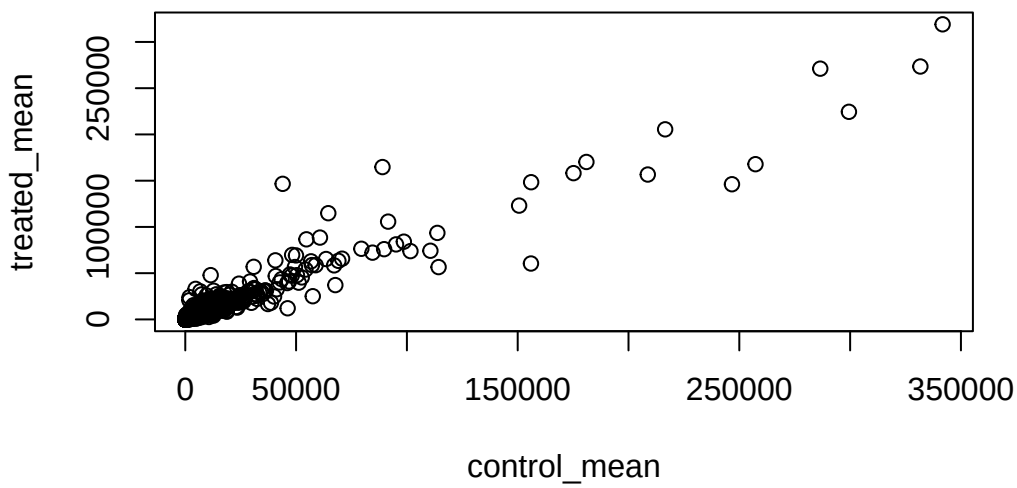
> Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called treated.mean)

```r
treated_ids <- metadata$id[metadata$dex == "treated"]
treated_counts <- counts[, treated_ids]
treated_mean <- rowMeans(treated_counts)
```

This gets the average treated expression for each gene. Next, we can make a scatter plot.

Q5 (a). Create a scatter plot showing the mean of the treated samples against the mean of the control samples. Your plot should look something like the following.

```
control_ids <- metadata$id[metadata$dex == "control"]
control_counts <- counts[, control_ids]
control_mean <- rowMeans(control_counts)
plot(control_mean, treated_mean)
```

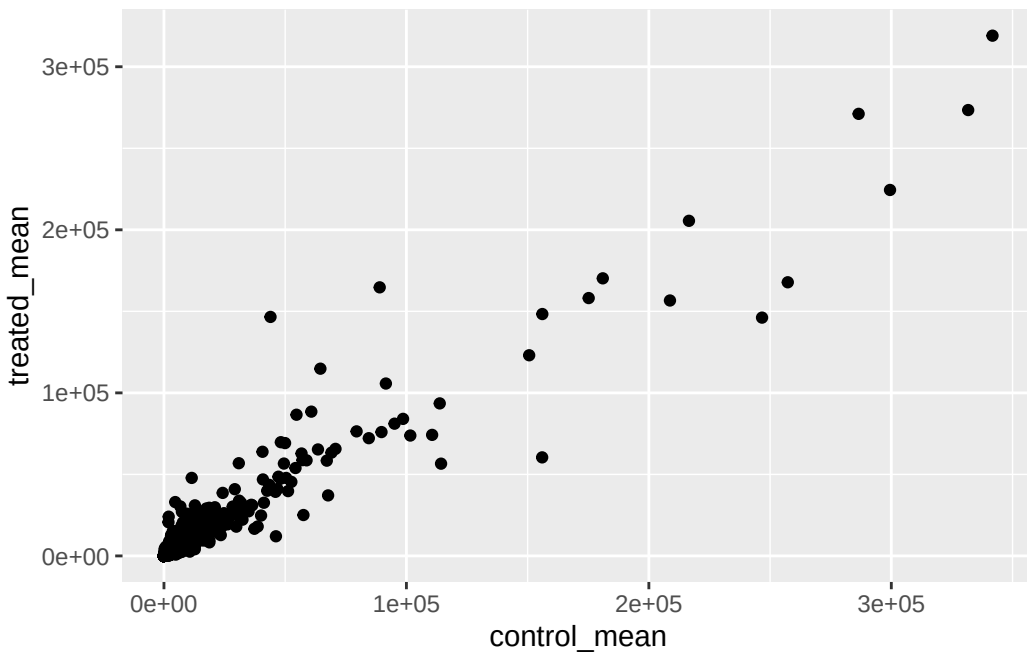


What this shows: - Each dot = one gene - If dots lie on diagonal = no change - If above diagonal = increased in treated - If below = decreased

Q5 (b).You could also use the ggplot2 package to make this figure producing the plot below. What geom_?() function would you use for this plot?

```
library(ggplot2)

control_ids <- metadata$id[metadata$dex == "control"]
control_counts <- counts[, control_ids]
control_mean <- rowMeans(control_counts)

treated_ids <- metadata$id[metadata$dex == "treated"]
treated_counts <- counts[, treated_ids]
treated_mean <- rowMeans(treated_counts)
```

```r
meancounts <- data.frame(control_mean, treated_mean)

ggplot(meancounts, aes(control_mean, treated_mean)) +
  geom_point()
```

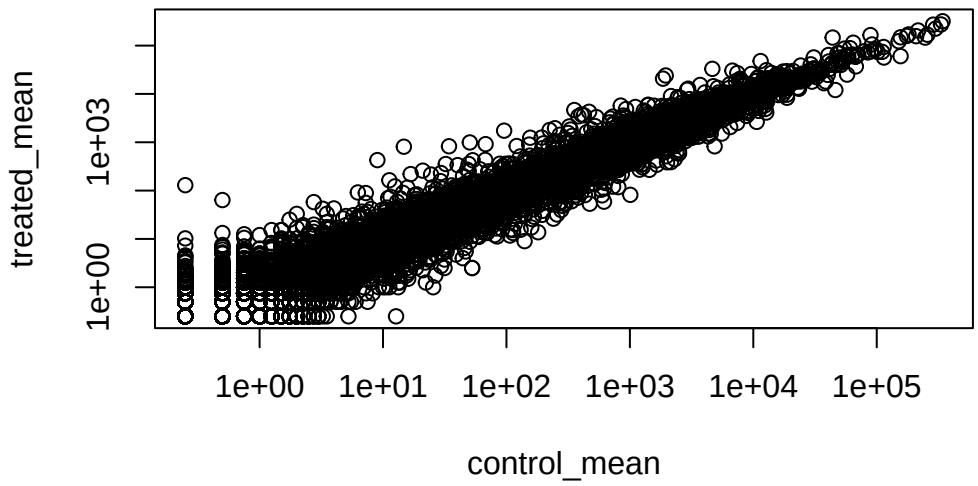


You must use `geom_point()`.

> Q6. Try plotting both axes on a log scale. What is the argument to plot() that allows you to do this?

```r
plot(control_mean, treated_mean, log = "xy")
```

```
Warning in xy.coords(x, y, xlabel, ylabel, log): 15032 x values <= 0 omitted
from logarithmic plot

Warning in xy.coords(x, y, xlabel, ylabel, log): 15281 y values <= 0 omitted
from logarithmic plot
```



`log = "xy"`, Gene expression ranges from very small to very large. The log scale spreads the points out.

We can find candidate differentially expressed genes by looking for genes with a large change between control and dex-treated samples. We usually look at the log2 of the fold change, because this has better mathematical properties.

Here we calculate log2foldchange, add it to our `meancounts` data.frame and inspect the results either with the *head()* or the *View()* function for example.

> Q7. What is the purpose of the arr.ind argument in the which() function call above? Why would we then take the first column of the output and need to call the unique() function?

```
head(which(meancounts[,1:2] == 0, arr.ind = TRUE))
```

```
                row col
ENSG00000000005   2   1
ENSG00000004848  65   1
ENSG00000004948  70   1
ENSG00000005001  73   1
ENSG00000006059 121   1
ENSG00000006071 123   1
```

`arr.ind = TRUE` tells R to return row AND column indices as a matrix. Without it, it will return a single index vector. We should use unique row indices because the same gene could have zero in both control and treated. We only want to remove each gene once.

A common threshold used for calling something differentially expressed is a log2(FoldChange) of greater than 2 or less than -2. Let's filter the dataset both ways to see how many genes are up or down-regulated.

> Q8. Using the up.ind vector above can you determine how many up regulated genes we have at the greater than 2 fc level?

```
meancounts$log2fc <- log2(meancounts$treated_mean /
                          meancounts$control_mean)
zero.vals <- which(meancounts[,1:2] == 0, arr.ind=TRUE)
to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
sum(mycounts$log2fc > 2)
```

```
[1] 250
```

Genes increased at least 4-fold.

> Q9. Using the down.ind vector above can you determine how many down regulated genes we have at the greater than 2 fc level?

```
sum(mycounts$log2fc < -2)
```

```
[1] 367
```

Genes decreased at least 4-fold.

> Q10. Do you trust these results? Why or why not?

No. There is no normalization, no statistical testing, no biological variability modeling, and it is just fold-change cutoff. This is overly simplistic.

Setting up for DESeq

Just like R packages from CRAN, you only need to install Bioconductor packages once (instructions here), then load them every time you start a new R session.

```r
library(DESeq2)
citation("DESeq2")
```

```
To cite package 'DESeq2' in publications use:

  Love, M.I., Huber, W., Anders, S. Moderated estimation of fold change
  and dispersion for RNA-seq data with DESeq2 Genome Biology 15(12):550
  (2014)

A BibTeX entry for LaTeX users is

  @Article{,
    title = {Moderated estimation of fold change and dispersion for RNA-
seq data with DESeq2},
    author = {Michael I. Love and Wolfgang Huber and Simon Anders},
    year = {2014},
    journal = {Genome Biology},
    doi = {10.1186/s13059-014-0550-8},
    volume = {15},
    issue = {12},
    pages = {550},
  }
```

Importing data Bioconductor software packages often define and use custom class objects for storing data. This helps to ensure that all the needed data for analysis (and the results) are available. DESeq works on a particular type of object called a DESeqDataSet. The DESeqDataSet is a single object that contains input values, intermediate calculations like how things are normalized, and all results of a differential expression analysis.

```r
dds <- DESeqDataSetFromMatrix(
  countData = counts,
  colData = metadata,
  design = ~ dex
)
```

```
converting counts to integer mode
```

```
Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

It tells DESeq2 to compare treated vs control.

This runs full statistical modeling: - Normalization - Dispersion estimation - Negative binomial modeling - Hypothesis testing

Now, we can extract results using `res <- results(dds)`… each gene has log2FoldChange, p-value, and adjusted p-value (padj)

We can also make a Volcano Plot:

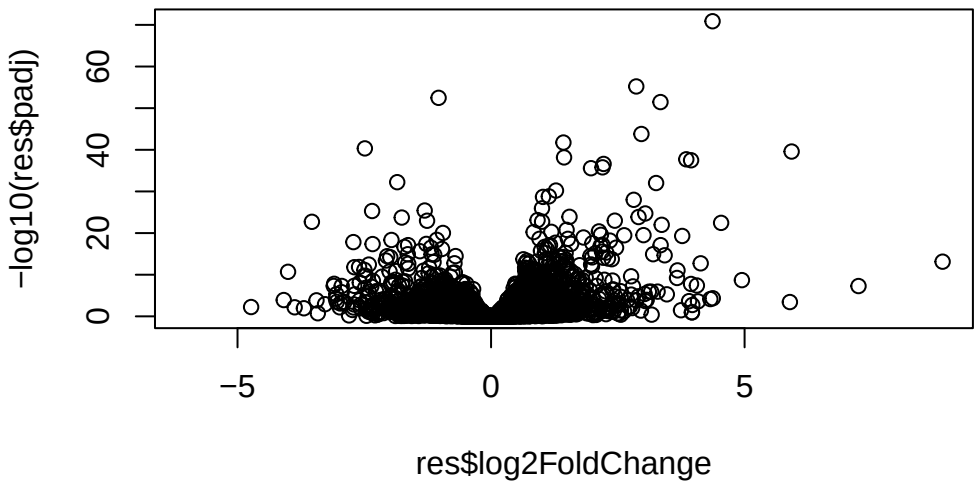
```
dds <- DESeqDataSetFromMatrix(
  countData = counts,
  colData = metadata,
  design = ~ dex
)
```

```
converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

```
dds <- DESeq(dds)
```

```
estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing
```

```
res <- results(dds)

plot(res$log2FoldChange,
     -log10(res$padj))
```



This shows X as size of change and Y as statistical significance. High + far right/left = important genes.